

IC MEASUREMENTS PROJECT REPORT

SENSOR TESTED: TCS34725

Test 0:

Analysis of data gathered in nominal condition (VDD = 3 V, TA = 25C, AGAIN = 16, AEN = 1).

(Room Temp: 25C (assumed))

Setting VDD=3A

```
dwf.FDwfDeviceCloseAll()
filter_flags = c_int(constants.enumfilterType.value | constants.enumfilterUSB.value)
device_count = c_int()
dwf.FDwfEnum(filter_flags, byref(device_count))

hdfw = c_int()
dwf.FDwfDeviceOpen(c_int(-1), byref(hdfw))
if hdfw.value == 0:
    raise RuntimeError("Failed to open device.")
print(hdfw.value)

dwf.FDwfAnalogIOReset(hdfw)
dwf.FDwfAnalogIOChannelNodeSet(hdfw, c_int(0), c_int(1), c_double(3.0))
dwf.FDwfAnalogIOChannelNodeSet(hdfw, c_int(0), c_int(0), c_double(1))
dwf.FDwfAnalogIOEnableSet(hdfw, c_int(1))
time.sleep(0.5)

vpos = c_double()
dwf.FDwfAnalogIOStatus(hdfw)
dwf.FDwfAnalogIOChannelNodeStatus(hdfw, c_int(0), c_int(1), byref(vpos))
print(f"Power: {vpos.value:.2f}V")
```

Setting AEN=1, AGAIN=16

```
write_register(ENABLE_REG, 0x01) #Power on / PON = 1
time.sleep(0.01)
write_register(ENABLE_REG, 0x03) #Now, also RGBC enable / PON = 1, AEN = 1
write_register(ATIME_REG, 0xF6) #To run for 24ms, upto a max count of 10,240
write_register(CONTROL_REG, 0x10) #Setting AGAIN=16

def read_colors():
    time.sleep(0.1)

    clear_low = read_register(CDATAH_REG)
    clear_high = read_register(CDATAH_REG)
    red_low = read_register(RDATAH_REG)
    red_high = read_register(RDATAH_REG)
    green_low = read_register(GDATAH_REG)
    green_high = read_register(GDATAH_REG)
    blue_low = read_register(BDATAH_REG)
    blue_high = read_register(BDATAH_REG)

    if all(val is not None for val in [clear_low, clear_high, red_low, red_high, green_low, green_high, blue_low, blue_high]):
        clear = (clear_high << 8) | clear_low
        red = (red_high << 8) | red_low
        green = (green_high << 8) | green_low
        blue = (blue_high << 8) | blue_low
        return {'clear': clear, 'red': red, 'green': green, 'blue': blue}
    return None
```

Outputs:

```
dwf.FDwfDeviceCloseAll()
filter_flags = c_int(constants.enumfilterType.value | constants.enumfilterUSB.value)
device_count = c_int()
dwf.FDwfEnum(filter_flags, byref(device_count))

hdfw = c_int()
dwf.FDwfDeviceOpen(c_int(-1), byref(hdfw))
if hdfw.value == 0:
    raise RuntimeError("Failed to open device.")
print(hdfw.value)

dwf.FDwfAnalogIOReset(hdfw)
dwf.FDwfAnalogIOChannelNodeSet(hdfw, c_int(0), c_int(1), c_double(3.0)) #setting VDD = 3V
dwf.FDwfAnalogIOChannelNodeSet(hdfw, c_int(0), c_int(0), c_double(1))
dwf.FDwfAnalogIOEnableSet(hdfw, c_int(1))
time.sleep(0.5)

vpos = c_double()
dwf.FDwfAnalogIOStatus(hdfw)
dwf.FDwfAnalogIOChannelNodeStatus(hdfw, c_int(0), c_int(1), byref(vpos))
print([f"Power: +{vpos.value:.2f}V"])
```

[5] ✓ 0.8s

... 1
Power: +3.01V

```
write_register(ENABLE_REG, 0x01) #Power on / PON = 1
time.sleep(0.01)
write_register(ENABLE_REG, 0x03) #Now, also RGBC enable / PON = 1, AEN = 1
write_register(ETIME_REG, 0xF6) #To run for 24ms, upto a max count of 10,240
write_register(CONTROL_REG, 0x10) #Setting AGAIN=16

def read_colors():
    time.sleep(0.1)

    clear_low = read_register(CDATAL_REG)
    clear_high = read_register(CDATAH_REG)
    red_low = read_register(RDATAL_REG)
    red_high = read_register(RDATAH_REG)
    green_low = read_register(GDATAL_REG)
    green_high = read_register(GDATAH_REG)
    blue_low = read_register(BDATAL_REG)
    blue_high = read_register(BDATAH_REG)

    if all(val is not None for val in [clear_low, clear_high, red_low, red_high, green_low, green_high, blue_low, blue_high]):
        clear = (clear_high <= 8) | clear_low
        red = (red_high <= 8) | red_low
        green = (green_high <= 8) | green_low
        blue = (blue_high <= 8) | blue_low
        return {'clear': clear, 'red': red, 'green': green, 'blue': blue}
    return None
```

1] ✓ 0.0s

```
write_register(CONTROL_REG,0)
for i in range(256):
    write_register(ATYPE_REG,255-i)
    colors = read_colors()

    print(f"Clear: {colors['clear']}, R: {colors['red']}, G: {colors['green']}, B: {colors['blue']}")

[38] ✓ 31.1s

...
Clear: 0, R: 0, G: 0, B: 0
Clear: 0, R: 0, G: 0, B: 0
Clear: 0, R: 0, G: 0, B: 0
Clear: 0, R: 0, G: 0, B: 0
Clear: 0, R: 0, G: 0, B: 0
Clear: 0, R: 0, G: 0, B: 0
Clear: 0, R: 0, G: 0, B: 0
Clear: 0, R: 0, G: 0, B: 0
Clear: 1, R: 0, G: 0, B: 0
Clear: 1, R: 0, G: 0, B: 0
Clear: 1, R: 0, G: 0, B: 0
Clear: 1, R: 0, G: 0, B: 0
Clear: 1, R: 0, G: 0, B: 0
Clear: 1, R: 0, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 1, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 0, B: 0
Clear: 2, R: 1, G: 1, B: 0
...
Clear: 20, R: 12, G: 6, B: 4
Clear: 20, R: 12, G: 6, B: 4
Clear: 20, R: 12, G: 6, B: 4
```

```
print("Settings applied. Stabilizing for 1 second...")
time.sleep(1.0)
print("Stabilizing done, now testing...")
print(" ")

clear_channel_samples = []
for i in range(1000):
    colors = read_colors()

    if colors:
        # Save the 'Clear' channel for statistical analysis
        clear_channel_samples.append(colors['clear'])

        # Print every 100th sample to avoid flooding the output window
        if i % 100 == 0:
            print(f"Sample {i}: Clear: {colors['clear']}, R: {colors['red']}, G: {colors['green']}, B: {colors['blue']}")

#ANALYSIS
if len(clear_channel_samples) > 0:
    mean_val = np.mean(clear_channel_samples)
    std_dev = np.std(clear_channel_samples)
    noise_percent = (std_dev / mean_val) * 100

    print("\n" + "="*40)
    print(f"ANALYSIS RESULT (N=1000)")
    print(f"Mean Count:      {mean_val:.2f}")
    print(f"Standard Dev:      {std_dev:.2f}")
    print(f"Noise (Jitter):      {noise_percent:.3f}%")
    print("="*40)
else:
    print("Error: No data collected.")

[30] ⚙ 22.9s
```

```
[30] ✓ 2m 1.4s
... Settings applied. Stabilizing for 1 second...
Stabilizing done, now testing...

Sample 0: Clear: 21, R: 13, G: 6, B: 4
Sample 100: Clear: 21, R: 13, G: 6, B: 4
Sample 200: Clear: 21, R: 13, G: 6, B: 4
Sample 300: Clear: 21, R: 13, G: 6, B: 4
Sample 400: Clear: 21, R: 13, G: 6, B: 4
Sample 500: Clear: 21, R: 13, G: 6, B: 4
Sample 600: Clear: 21, R: 13, G: 6, B: 4
Sample 700: Clear: 21, R: 13, G: 6, B: 4
Sample 800: Clear: 21, R: 13, G: 6, B: 4
Sample 900: Clear: 21, R: 13, G: 6, B: 4

=====
ANALYSIS RESULT (N=1000)
Mean Count:      21.00
Standard Dev:    0.00
Noise (Jitter):  0.000%
=====
```

Since, we are getting the output to be exact same even after running the simulation for 1000 times, this means that the sensor is not updating the value at all. But, if we check the starting samples of the sensor, we can see that the sensor is working and it is updating the value as it is running. So, we can conclude that:

Conclusion: The data is consistent. The sensor is reading the exact same light level in both tests. It is not stuck on a random number; it is accurately reporting a very low light level.

Thus, this proves that the sensor is working fine, but the conditions in which the sensor is present right now are very dark/no external light source available.

****Testing the same data after 12 hrs.**

```
write_register(CONTROL_REG,0)
for i in range(256):
    write_register(ATIME_REG,255-i)
    colors = read_colors()

    print(f"Clear: {colors['clear']}, R: {colors['red']}, G: {colors['green']}, B: {colors['blue']}")
```

(17) ✓ 31% Python

... Clear: 2, R: 1, G: 1, B: 1
Clear: 4, R: 1, G: 2, B: 1
Clear: 6, R: 2, G: 2, B: 2
Clear: 8, R: 3, G: 3, B: 3
Clear: 10, R: 4, G: 4, B: 3
Clear: 12, R: 4, G: 5, B: 4
Clear: 14, R: 5, G: 5, B: 5
Clear: 16, R: 6, G: 6, B: 5
Clear: 18, R: 6, G: 7, B: 6
Clear: 20, R: 7, G: 8, B: 7
Clear: 21, R: 8, G: 9, B: 8
Clear: 23, R: 9, G: 9, B: 8
Clear: 25, R: 9, G: 10, B: 9
Clear: 27, R: 10, G: 11, B: 10
Clear: 29, R: 11, G: 12, B: 10
Clear: 31, R: 12, G: 13, B: 11
Clear: 33, R: 12, G: 13, B: 12
Clear: 35, R: 13, G: 14, B: 12
Clear: 37, R: 14, G: 15, B: 13
Clear: 39, R: 15, G: 16, B: 14
Clear: 41, R: 15, G: 17, B: 14
Clear: 43, R: 16, G: 17, B: 15
Clear: 45, R: 17, G: 18, B: 16
Clear: 47, R: 17, G: 19, B: 17
Clear: 49, R: 18, G: 20, B: 17
...
Clear: 482, R: 180, G: 195, B: 170
Clear: 482, R: 180, G: 199, B: 173
Clear: 492, R: 184, G: 199, B: 173
Clear: 492, R: 184, G: 199, B: 173

```
Settings applied. Stabilizing for 1 second...
Stabilizing done, now testing...

Sample 0: Clear: 321, R: 119, G: 131, B: 112
Sample 100: Clear: 322, R: 120, G: 131, B: 113
Sample 200: Clear: 323, R: 120, G: 131, B: 113
Sample 300: Clear: 324, R: 120, G: 131, B: 113
Sample 400: Clear: 324, R: 120, G: 132, B: 113
Sample 500: Clear: 324, R: 120, G: 132, B: 113
Sample 600: Clear: 323, R: 120, G: 131, B: 113
Sample 700: Clear: 322, R: 120, G: 131, B: 112
Sample 800: Clear: 322, R: 120, G: 131, B: 113
Sample 900: Clear: 322, R: 119, G: 131, B: 113

=====
ANALYSIS RESULT (N=1000)
Mean Count:      322.82
Standard Dev:    0.86
Noise (Jitter):  0.265%
=====
```

As we can see now, the values of light are much sharper now and we can clearly notice that there is a slight standard deviation and jitter in the data values, which tells us that the sensor is reading values correctly.

Test 1.0:

Verify sensor reading counts proportional to atime for all channels

INTEGRATION TIME: The RGBC timing register controls the internal integration time of the RGBC clear and IR channel ADCs in 2.4-ms increments.

The registers for programming the integration and wait times are a 2's complement values. The actual time can be calculated as follows:

$$ATIME = 256 - \text{Integration Time} / 2.4 \text{ ms}$$

Inversely, the time can be calculated from the register value as follows:

$$\text{Integration Time} = 2.4 \text{ ms} \times (256 - ATIME)$$

For example,

If a 100-ms integration time is needed, the device needs to be programmed to:

$$256 - (100 / 2.4) = 256 - 42 = 214 = 0xD6$$

Conversely, the programmed value of 0xC0 would correspond to:

$$(256 - 0xC0) \times 2.4 = 64 \times 2.4 = 154 \text{ ms.}$$

Max RGBC Count = $(256 - ATIME) \times 1024$ up to a maximum of 65535.

//I'm increasing the gain here so that we can notice better difference in the sensor values.

AGAIN=0x03

//Also, we need "wait time" here in order to give sensor time to collect light and give result. The wait time should be dynamic because for smaller cases (eg: ATIME=255) we need small duration of wait time, but for higher ATIME cases we need higher wait times accordingly. So, we calculate wait time proportional to the integration time.

Wait Time = Integration time + small buffer time (1ms, for safety)

Note: This wait time is for python, not the sensor so that python waits for sometime before sending the request to sensor and so that sensor can take its time collecting each output.

```

write_register(CONTROL_REG,3)
for i in range(256):
    atime_val = 255-i
    write_register(ATYPE_REG,atime_val)
    colors = read_colors()

    # Calculating sleep time for python
    needed_time = (256 - atime_val) * 0.0024
    time.sleep(needed_time)

print(f"ATYPE: {255-i}, Clear: {colors['clear']}, R: {colors['red']}, G: {colors['green']}, B: {colors['blue']}")

```

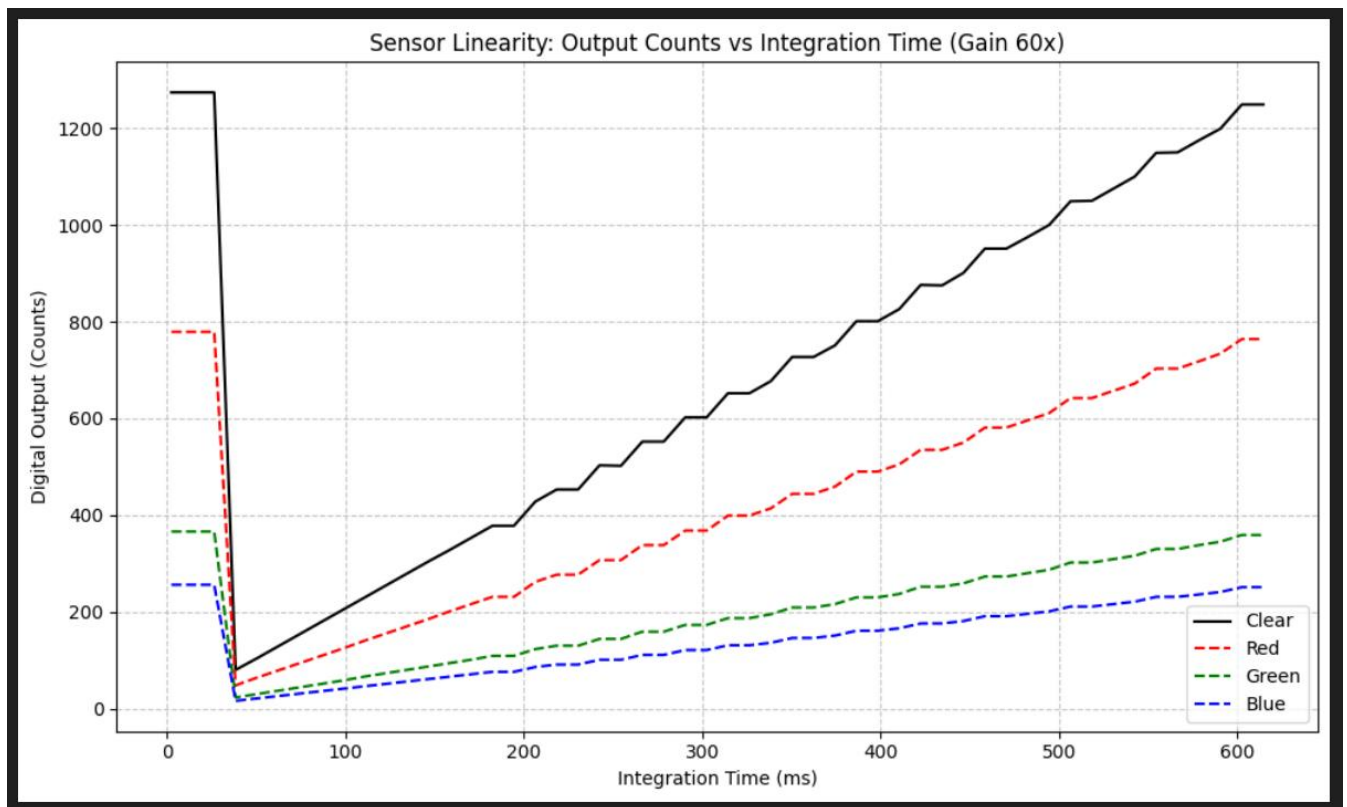
[16] 23.5s

```

... ATIME: 255, Clear: 5, R: 3, G: 1, B: 1
    ATIME: 254, Clear: 10, R: 6, G: 3, B: 2
    ATIME: 253, Clear: 15, R: 9, G: 4, B: 3
    ATIME: 252, Clear: 20, R: 12, G: 6, B: 4
    ATIME: 251, Clear: 25, R: 15, G: 7, B: 5
    ATIME: 250, Clear: 30, R: 18, G: 8, B: 6
    ATIME: 249, Clear: 35, R: 21, G: 10, B: 7
    ATIME: 248, Clear: 40, R: 24, G: 11, B: 8
    ATIME: 247, Clear: 45, R: 27, G: 13, B: 9
    ATIME: 246, Clear: 50, R: 30, G: 14, B: 10
    ATIME: 245, Clear: 55, R: 33, G: 16, B: 11
    ATIME: 244, Clear: 60, R: 36, G: 17, B: 12
    ATIME: 243, Clear: 64, R: 39, G: 18, B: 13
    ATIME: 242, Clear: 69, R: 42, G: 20, B: 14
    ATIME: 241, Clear: 74, R: 45, G: 21, B: 15
    ATIME: 240, Clear: 79, R: 48, G: 23, B: 16
    ATIME: 239, Clear: 84, R: 51, G: 24, B: 17
    ATIME: 238, Clear: 89, R: 55, G: 26, B: 18
    ATIME: 237, Clear: 94, R: 58, G: 27, B: 19
    ATIME: 236, Clear: 99, R: 61, G: 28, B: 20
    ATIME: 235, Clear: 104, R: 64, G: 30, B: 21
    ATIME: 234, Clear: 109, R: 67, G: 31, B: 22
    ATIME: 233, Clear: 114, R: 70, G: 33, B: 23
    ATIME: 232, Clear: 119, R: 73, G: 34, B: 24
    ATIME: 231, Clear: 124, R: 76, G: 36, B: 25
    ...
    ATIME: 3, Clear: 1254, R: 767, G: 360, B: 252
    ATIME: 2, Clear: 1259, R: 770, G: 362, B: 253
    ATIME: 1, Clear: 1259, R: 770, G: 362, B: 253
    ATIME: 0, Clear: 1264, R: 773, G: 363, B: 254

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)



The bar in the start of the graph means the sensor was sitting idle in the start form the previous run. After that, the sensor values drops and they start taking the values.

From this graph, we can easily justify that:

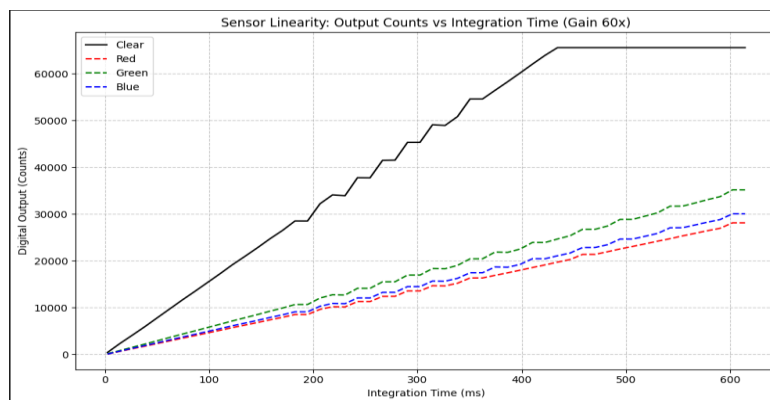
- The output is linear with respect to the ATIME.
- The output is having a 'staircase effect', which is what happens due to the quantization of light, or in simple words, the sensor is taking input during this time, but hasn't tested it yet.

Thus, we can say that this test was a definite success!

****Reading again 12 hrs later**

```
[21] ✓ 1m 50.0s
... ATIME: 255, Clear: 126, R: 47, G: 51, B: 44
    ATIME: 254, Clear: 252, R: 93, G: 103, B: 88
    ATIME: 253, Clear: 379, R: 141, G: 154, B: 132
    ATIME: 252, Clear: 505, R: 187, G: 206, B: 176
    ATIME: 251, Clear: 631, R: 234, G: 257, B: 220
    ATIME: 250, Clear: 757, R: 281, G: 309, B: 264
    ATIME: 249, Clear: 883, R: 328, G: 360, B: 308
    ATIME: 248, Clear: 1010, R: 375, G: 411, B: 352
    ATIME: 247, Clear: 1135, R: 422, G: 463, B: 397
    ATIME: 246, Clear: 1262, R: 469, G: 514, B: 441
    ATIME: 245, Clear: 1388, R: 516, G: 566, B: 485
    ATIME: 244, Clear: 1515, R: 563, G: 617, B: 529
    ATIME: 243, Clear: 1640, R: 609, G: 669, B: 573
    ATIME: 242, Clear: 1767, R: 656, G: 720, B: 617
    ATIME: 241, Clear: 1893, R: 703, G: 772, B: 661
    ATIME: 240, Clear: 2018, R: 750, G: 823, B: 705
    ATIME: 239, Clear: 2145, R: 797, G: 874, B: 749
    ATIME: 238, Clear: 2270, R: 843, G: 925, B: 793
    ATIME: 237, Clear: 2397, R: 890, G: 977, B: 837
    ATIME: 236, Clear: 2523, R: 937, G: 1029, B: 881
    ATIME: 235, Clear: 2650, R: 984, G: 1080, B: 925
    ATIME: 234, Clear: 2776, R: 1031, G: 1132, B: 969
    ATIME: 233, Clear: 2902, R: 1078, G: 1183, B: 1014
    ATIME: 232, Clear: 3028, R: 1125, G: 1234, B: 1057
    ATIME: 231, Clear: 3153, R: 1171, G: 1285, B: 1101
    ...
    ATIME: 3, Clear: 65535, R: 28191, G: 35240, B: 30872
    ATIME: 2, Clear: 65535, R: 28315, G: 35398, B: 30206
    ATIME: 1, Clear: 65535, R: 28408, G: 35518, B: 30307
    ATIME: 0, Clear: 65535, R: 28530, G: 35815, B: 30569
    Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
... --- Test 1.0: Linearity Data Collection ---
    Time: 2.4ms -> Clear: 437
    Time: 14.4ms -> Clear: 2359
    Time: 26.4ms -> Clear: 4148
    Time: 38.4ms -> Clear: 5968
    Time: 50.4ms -> Clear: 7878
    Time: 62.4ms -> Clear: 9777
    Time: 74.4ms -> Clear: 11675
    Time: 86.4ms -> Clear: 13499
    Time: 98.4ms -> Clear: 15374
    Time: 110.4ms -> Clear: 17243
    Time: 122.4ms -> Clear: 19180
    Time: 134.4ms -> Clear: 20972
    Time: 146.4ms -> Clear: 22802
    Time: 158.4ms -> Clear: 24716
    Time: 170.4ms -> Clear: 26691
    Time: 182.4ms -> Clear: 28514
    Time: 194.4ms -> Clear: 28505
    Time: 206.4ms -> Clear: 32192
    Time: 218.4ms -> Clear: 34882
    Time: 230.4ms -> Clear: 33905
    Time: 242.4ms -> Clear: 37756
    Time: 254.4ms -> Clear: 37730
    Time: 266.4ms -> Clear: 41453
    Time: 278.4ms -> Clear: 41499
    ...
    Time: 602.4ms -> Clear: 65535
    Time: 614.4ms -> Clear: 65535
    Generating Plot...
    Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```



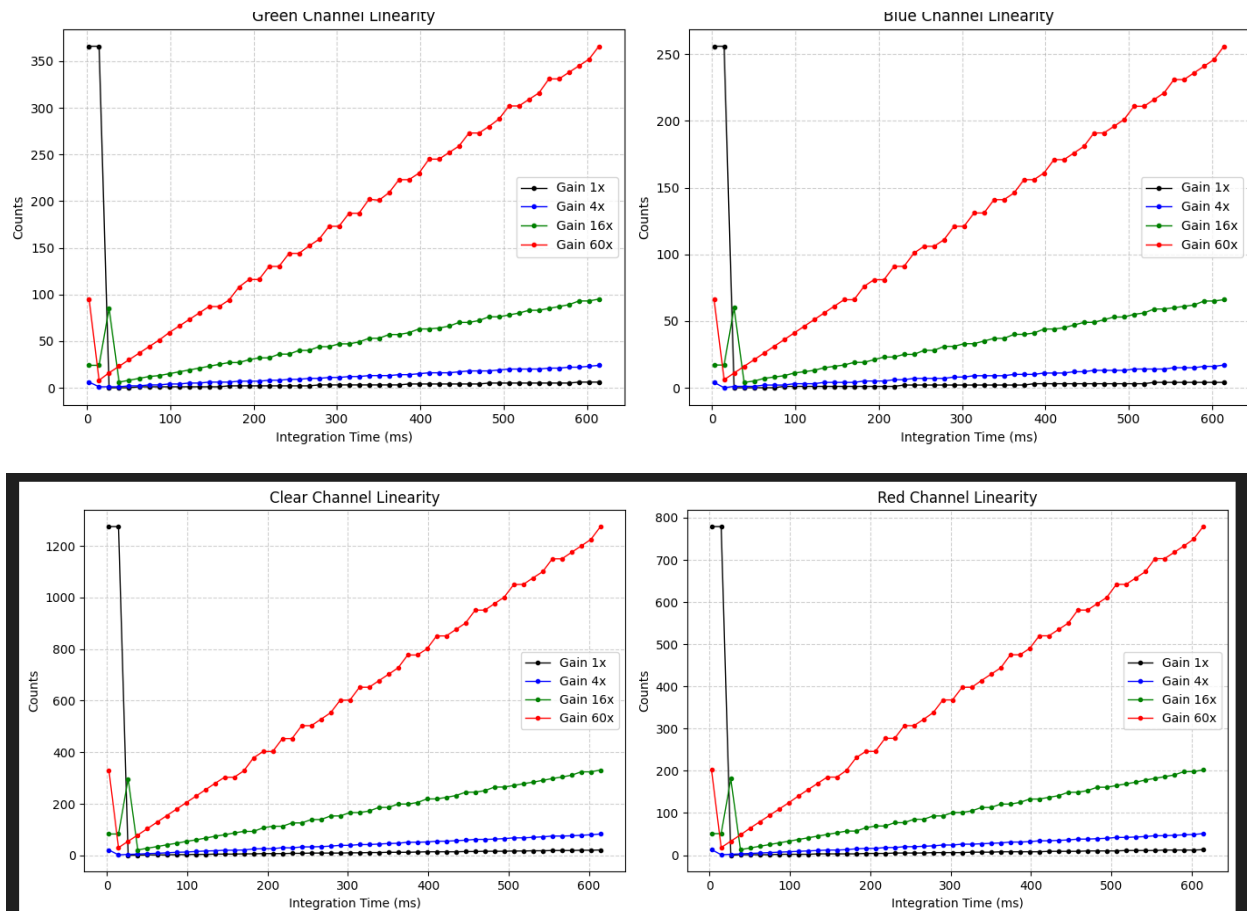
This proves that our earlier evaluation was correct, just the data values are what is changed as of now.

****Staircase effect in the graph due to “time.sleep” that we used.**

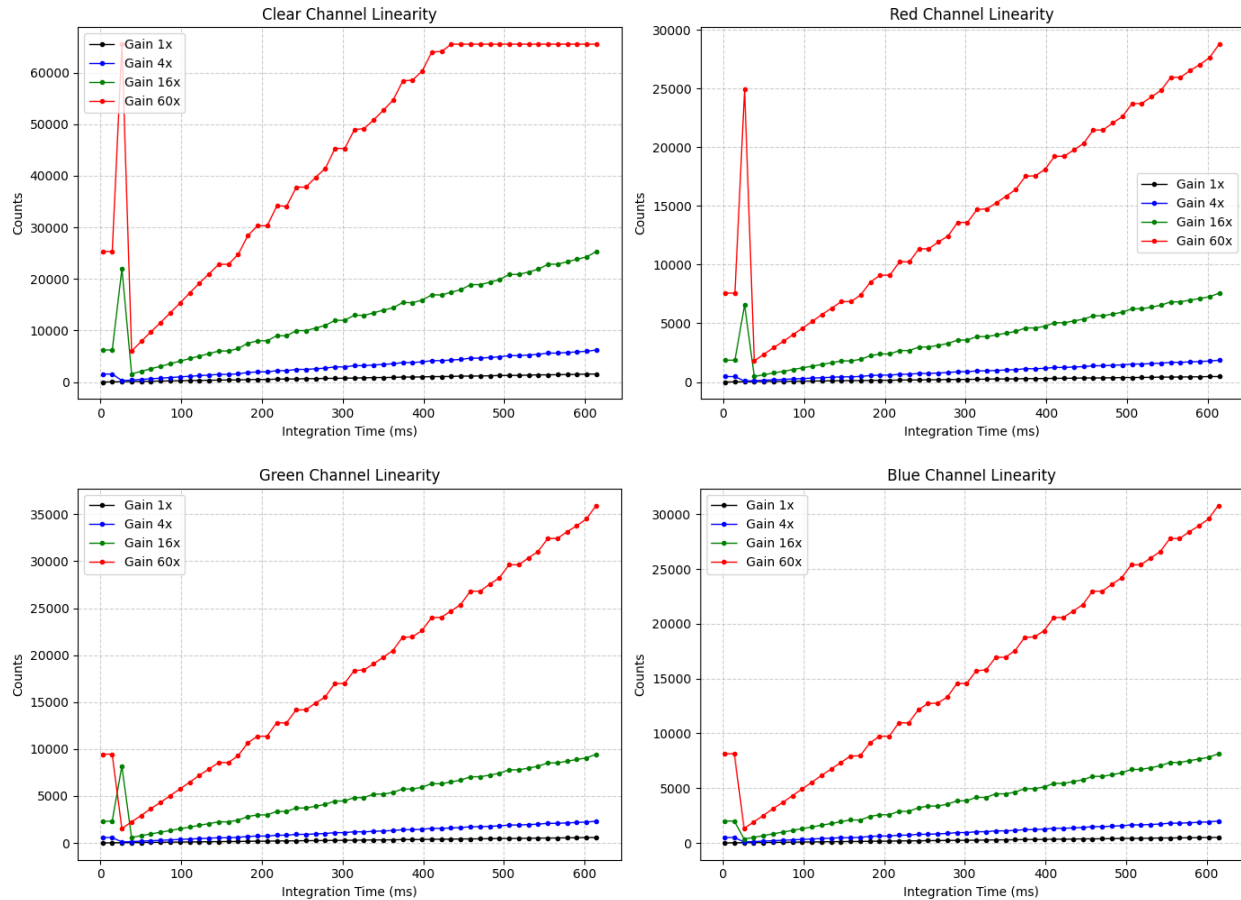
Test 1.1:

Verify sensor reading counts proportional to gain for all channels.

Just change the Gain values 4 times and plot atime vs 'color' with output for each gain value in a single plot in order to observe their outputs.



**Testing again 12 hrs. later.



Again, the result is changed, but the output is better now. But still our analysis was correct even before.

***When you first power on the sensor and immediately start a sweep, the very first read might happen before the first controlled integration cycle (at 2.4ms) is complete.*

***If the sensor was already powered and sitting under light, the registers might already be holding a saturated value from a previous state.*

Test 2.0:

Characterize the effect of wait time on sensor reading counts.

Setting up WTIME_REG

```
DEVICE_ADDR = 0x52
ENABLE_REG = 0x80
ATIME_REG = 0x81
WTIME_REG = 0x83
CONTROL_REG = 0x8F
CDATAL_REG = 0x94
CDATAH_REG = 0x95
RDATAH_REG = 0x96
RDATAH_REG = 0x97
GDATAL_REG = 0x98
GDATAL_REG = 0x99
BDATAL_REG = 0x9A
BDATAH_REG = 0x9B
```

```
write_register(ENABLE_REG, 0x01) #Power on / PON = 1
time.sleep(0.01)
write_register(ENABLE_REG, 0x0B) #Now, also RGBC enable / PON = 1, AEN = 1, WEN = 1
write_register(ATIME_REG, 0xF6) #To run for 24ms, upto a max count of 10,240
write_register(CONTROL_REG, 0x02) #Setting AGAIN=16
write_register(WTIME_REG, 0xFF) #Resetting the wtime register
```

✓ 0.0s

For output right now, I'm fixing the ATIME value and changing the WTIME value

```
#Test case 1, Setting ATIME = 0xF6
write_register(CONTROL_REG, 3)
write_register(ATIME_REG, 0xF6)

wtime_val = [0xFF, 0xAB, 0x00]

for wtime in wtime_val:
    wait_time = (256-wtime)*2.4
    write_register(WTIME_REG, wtime)
    integration_time = (256-246)*2.4
    total_time = (integration_time + wait_time)/1000.0
    time.sleep(total_time)
    colors = read_colors()

    print(f"WAIT_REG: {wtime}, Clear: {colors['clear']}, R: {colors['red']}, G: {colors['green']}, B: {colors['blue']}")
```

✓ 1.2s

```
WAIT_REG: 255, Clear: 3855, R: 1156, G: 1444, B: 1232
WAIT_REG: 171, Clear: 3646, R: 1100, G: 1370, B: 1176
WAIT_REG: 0, Clear: 3869, R: 1160, G: 1449, B: 1243
```

```
> v
#Test case 2, Setting ATIME = 0xC0
write_register(CONTROL_REG,3)
write_register(ATYPE_REG,0xC0)

wtime_val = [0xFF, 0xAB, 0x00]

for wtime in wtime_val:
    wait_time = (256-wtime)*2.4
    write_register(WTIME_REG, wtime)
    integration_time = (256-246)*2.4
    total_time = (integration_time + wait_time)/1000.0
    time.sleep(total_time)
    colors = read_colors()

    print(f"WAIT_REG: {wtime}, Clear: {colors['clear']}, R: {colors['red']}, G: {colors['green']}, B: {colors['blue']}")

[41] ✓ 12s
... WAIT_REG: 255, Clear: 3720, R: 1125, G: 1397, B: 1202
    WAIT_REG: 171, Clear: 24431, R: 7372, G: 9171, B: 7878
    WAIT_REG: 0, Clear: 24475, R: 7382, G: 9188, B: 7892
```

As we can notice here, the wait time is not having any effect on the sensor reading, despite reading the values for different ATIME values.

Test 2.1:

Verify sensor reading counts [jitter] vs gain and vs atime.

For this, we need to get the output jitter (noise) for:

- Output vs gain (set default ATIME), and
- Output vs ATIME (set default gain)

We will calculate 100 samples for each outcome and test their jitter values.

Output vs Gain

```
print("---- Part 1: Data Collection (Jitter vs Gain) ----")

# Set Fixed ATIME (~10ms) for fair comparison
fixed_atime = 0x05
write_register(ATYPE_REG, fixed_atime)
integ_ms = (256 - fixed_atime) * 2.4

# Define Gains
gains = {0x00: 1, 0x01: 4, 0x02: 16, 0x03: 64}
results_gain = []

print(f"Fixed Time: {integ_ms:.1f}ms. Collecting 100 samples per Gain...")

# 2. SWEEP LOOP
for reg, mult in gains.items():
    write_register(CONTROL_REG, reg)
    time.sleep(0.2) # Stabilize

    samples = []
    # Collect 100 samples
    for i in range(100):
        c = read_colors()
        if i%10==0: print(f"GAIN = {mult}, Clear = {c['clear']}, Red = {c['red']}, Green = {c['green']}, Blue = {c['blue']}")
        if c: samples.append(c['clear'])
        time.sleep(0.005) # Small buffer

    # Calculate Stats
    if samples:
        mean = np.mean(samples)
        std = np.std(samples)
        jitter_pct = (std / mean * 100) if mean > 0 else 0

        print(f" Gain {mult:2d}x: Mean={mean:7.1f} | StdDev={std:5.2f} | Jitter={jitter_pct:5.3f}%")
        print("\n"+"*"*40)
        results_gain.append({'Gain': mult, 'Mean': mean, 'StdDev': std, 'Jitter_Pct': jitter_pct})

print("Part 1 Complete.")
```

```

1 |--- Part 1: Data Collection (Jitter vs Gain) ---
2 Fixed Time: 103.2ms. Collecting 100 samples per Gain...
3 GAIN = 1, Clear = 283, Red = 86, Green = 105, Blue = 91
4 GAIN = 1, Clear = 281, Red = 86, Green = 105, Blue = 91
5 GAIN = 1, Clear = 284, Red = 86, Green = 105, Blue = 92
6 GAIN = 1, Clear = 285, Red = 87, Green = 106, Blue = 92
7 GAIN = 1, Clear = 283, Red = 86, Green = 105, Blue = 92
8 GAIN = 1, Clear = 281, Red = 86, Green = 105, Blue = 91
9 GAIN = 1, Clear = 282, Red = 86, Green = 105, Blue = 92
10 GAIN = 1, Clear = 285, Red = 87, Green = 106, Blue = 92
11 GAIN = 1, Clear = 283, Red = 86, Green = 105, Blue = 92
12 GAIN = 1, Clear = 281, Red = 86, Green = 105, Blue = 91
13 | Gain 1x: Mean= 282.9 | StdDev= 1.46 | Jitter=0.518%
14
15 =====
16 GAIN = 4, Clear = 1106, Red = 337, Green = 413, Blue = 357
17 GAIN = 4, Clear = 1117, Red = 340, Green = 416, Blue = 360
18 GAIN = 4, Clear = 1121, Red = 341, Green = 418, Blue = 361
19 GAIN = 4, Clear = 1114, Red = 339, Green = 415, Blue = 359
20 GAIN = 4, Clear = 1106, Red = 337, Green = 413, Blue = 356
21 GAIN = 4, Clear = 1113, Red = 339, Green = 415, Blue = 358
22 GAIN = 4, Clear = 1121, Red = 341, Green = 418, Blue = 361
23 GAIN = 4, Clear = 1117, Red = 340, Green = 416, Blue = 360
24 GAIN = 4, Clear = 1105, Red = 337, Green = 412, Blue = 356
25 GAIN = 4, Clear = 1108, Red = 337, Green = 414, Blue = 357
26 | Gain 4x: Mean= 1113.1 | StdDev= 5.71 | Jitter=0.513%
27
28 =====
29 GAIN = 16, Clear = 4514, Red = 1375, Green = 1687, Blue = 1456
30 GAIN = 16, Clear = 4551, Red = 1385, Green = 1700, Blue = 1467
31 GAIN = 16, Clear = 4537, Red = 1381, Green = 1695, Blue = 1463
32 GAIN = 16, Clear = 4503, Red = 1372, Green = 1683, Blue = 1452
33 GAIN = 16, Clear = 4488, Red = 1368, Green = 1678, Blue = 1448
34 GAIN = 16, Clear = 4531, Red = 1380, Green = 1693, Blue = 1461
35 GAIN = 16, Clear = 4558, Red = 1386, Green = 1702, Blue = 1468
36 GAIN = 16, Clear = 4533, Red = 1380, Green = 1694, Blue = 1461
37 GAIN = 16, Clear = 4495, Red = 1370, Green = 1680, Blue = 1450
38 GAIN = 16, Clear = 4504, Red = 1373, Green = 1681, Blue = 1450
39 | Gain 16x: Mean= 4522.0 | StdDev=23.28 | Jitter=0.515%
40
41 =====
42 GAIN = 64, Clear = 17079, Red = 5226, Green = 6416, Blue = 5505
43 GAIN = 64, Clear = 17246, Red = 5271, Green = 6475, Blue = 5554
44 GAIN = 64, Clear = 17223, Red = 5264, Green = 6467, Blue = 5558
45 GAIN = 64, Clear = 17102, Red = 5233, Green = 6424, Blue = 5511
46 GAIN = 64, Clear = 17019, Red = 5211, Green = 6393, Blue = 5485
47 GAIN = 64, Clear = 17157, Red = 5248, Green = 6425, Blue = 5511
48 GAIN = 64, Clear = 17268, Red = 5278, Green = 6483, Blue = 5561
49 GAIN = 64, Clear = 17192, Red = 5258, Green = 6457, Blue = 5538
50 GAIN = 64, Clear = 17022, Red = 5221, Green = 6405, Blue = 5494
51 GAIN = 64, Clear = 17049, Red = 5219, Green = 6404, Blue = 5492
52 | Gain 64x: Mean=17139.4 | StdDev=89.55 | Jitter=0.522%
53
54 =====
55 Part 1 Complete.

```

As we can observe from the output above, there is a slight jitter of 0.5% in case of each gain value. This proves that despite the gain, the sensor readings are accurate and have only a 0.5% of noise in them.

Output vs ATIME

```
print("---- Part 2: Data Collection (Jitter vs ATIME) ----")

# 1. SETUP
# Set Fixed Gain to 64x (0x03) to ensure noise is visible
fixed_gain_reg = 0x03
write_register(CONTROL_REG, fixed_gain_reg)

# Define ATIME Steps: 2.4ms, 24ms, 101ms, 154ms, 614ms
atime_steps = [0xFF, 0xF6, 0xD5, 0xC0, 0x00]
results_time = []

print(f"Fixed Gain: 60x. Collecting 100 samples per ATIME...")

# 2. SWEEP LOOP
for atime_val in atime_steps:
    write_register(ATIME_REG, atime_val)

    # Calculate necessary wait per sample
    time_ms = (256 - atime_val) * 2.4
    wait_sec = (time_ms / 1000.0) + 0.01

    samples = []
    # Collect 100 samples
    for _ in range(100):
        time.sleep(wait_sec) # Must wait for integration cycle
        c = read_colors()
        if _ % 10 == 0: print(f"ATIME = {255 - atime_val}, Clear = {c['clear']}, Red = {c['red']}, Green = {c['green']}, Blue = {c['blue']}")
        if c: samples.append(c['clear'])

    # Calculate Stats
    if samples:
        mean = np.mean(samples)
        std = np.std(samples)
        jitter_pct = (std / mean * 100) if mean > 0 else 0

        print(f"   Time {time_ms:5.1f}ms: Mean={mean:7.1f} | StdDev={std:5.2f} | Jitter={jitter_pct:5.3f}%")
        print("\n"+"*" * 40)
        results_time.append({'Time_ms': time_ms, 'Mean': mean, 'StdDev': std, 'Jitter_Pct': jitter_pct})

print("Part 2 Complete.")
```

```
... --- Part 2: Data Collection (Jitter vs ATIME) ---
Fixed Gain: 60x. Collecting 100 samples per ATIME...
ATIME = 0, Clear = 411, Red = 145, Green = 180, Blue = 124
ATIME = 0, Clear = 424, Red = 129, Green = 180, Blue = 120
ATIME = 0, Clear = 313, Red = 132, Green = 178, Blue = 152
ATIME = 0, Clear = 320, Red = 101, Green = 168, Blue = 150
ATIME = 0, Clear = 332, Red = 104, Green = 172, Blue = 146
ATIME = 0, Clear = 344, Red = 107, Green = 176, Blue = 143
ATIME = 0, Clear = 357, Red = 110, Green = 178, Blue = 139
ATIME = 0, Clear = 370, Red = 114, Green = 179, Blue = 136
ATIME = 0, Clear = 484, Red = 145, Green = 154, Blue = 101
ATIME = 0, Clear = 485, Red = 145, Green = 151, Blue = 101
   Time   2.4ms: Mean= 398.0 | StdDev=62.35 | Jitter=15.667%

=====
ATIME = 9, Clear = 3902, Red = 1199, Green = 1467, Blue = 1254
ATIME = 9, Clear = 4059, Red = 1249, Green = 1535, Blue = 1316
ATIME = 9, Clear = 3933, Red = 1207, Green = 1479, Blue = 1269
ATIME = 9, Clear = 4029, Red = 1233, Green = 1513, Blue = 1308
ATIME = 9, Clear = 3994, Red = 1214, Green = 1487, Blue = 1276
ATIME = 9, Clear = 4007, Red = 1227, Green = 1505, Blue = 1291
ATIME = 9, Clear = 4014, Red = 1229, Green = 1507, Blue = 1281
ATIME = 9, Clear = 3954, Red = 1212, Green = 1500, Blue = 1287
ATIME = 9, Clear = 3998, Red = 1223, Green = 1501, Blue = 1289
ATIME = 9, Clear = 3949, Red = 1210, Green = 1484, Blue = 1274
   Time  24.0ms: Mean= 3990.8 | StdDev=89.89 | Jitter=2.252%

=====
ATIME = 42, Clear = 17252, Red = 5273, Green = 6473, Blue = 5557
ATIME = 42, Clear = 16927, Red = 5187, Green = 6363, Blue = 5465
ATIME = 42, Clear = 17139, Red = 5240, Green = 6436, Blue = 5529
ATIME = 42, Clear = 16917, Red = 5188, Green = 6359, Blue = 5462
ATIME = 42, Clear = 17132, Red = 5248, Green = 6436, Blue = 5525
ATIME = 42, Clear = 16948, Red = 5199, Green = 6371, Blue = 5469
ATIME = 42, Clear = 17147, Red = 5251, Green = 6438, Blue = 5524
ATIME = 42, Clear = 16959, Red = 5202, Green = 6373, Blue = 5473
ATIME = 42, Clear = 17226, Red = 5275, Green = 6470, Blue = 5552
ATIME = 42, Clear = 16979, Red = 5209, Green = 6381, Blue = 5479
   Time 103.2ms: Mean=17050.9 | StdDev=94.44 | Jitter=0.554%

=====
ATIME = 63, Clear = 25446, Red = 7796, Green = 9559, Blue = 8206
ATIME = 63, Clear = 25323, Red = 7763, Green = 9515, Blue = 8168
ATIME = 63, Clear = 25352, Red = 7771, Green = 9525, Blue = 8178
ATIME = 63, Clear = 25519, Red = 7817, Green = 9587, Blue = 8228
ATIME = 63, Clear = 25511, Red = 7813, Green = 9583, Blue = 8228
ATIME = 63, Clear = 25344, Red = 7773, Green = 9524, Blue = 8175
ATIME = 63, Clear = 25383, Red = 7786, Green = 9539, Blue = 8187
ATIME = 63, Clear = 25521, Red = 7825, Green = 9586, Blue = 8227
ATIME = 63, Clear = 25449, Red = 7802, Green = 9564, Blue = 8208
ATIME = 63, Clear = 25376, Red = 7783, Green = 9534, Blue = 8184
   Time 153.6ms: Mean=25426.0 | StdDev=67.09 | Jitter=0.264%
```

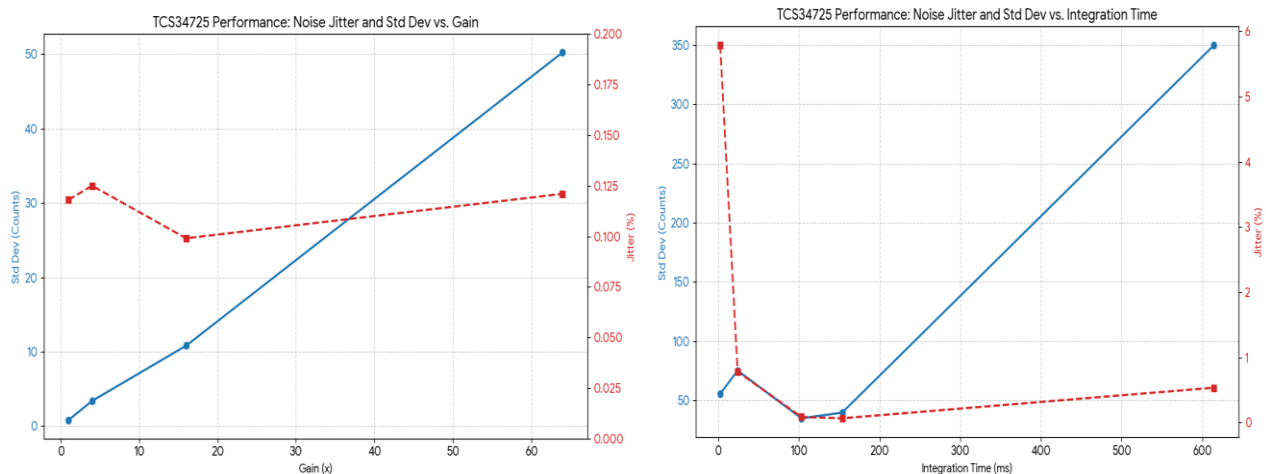
```

ATIME = 255, Clear = 65535, Red = 31203, Green = 38228, Blue = 32813
ATIME = 255, Clear = 65535, Red = 31166, Green = 38210, Blue = 32807
ATIME = 255, Clear = 65535, Red = 31171, Green = 38202, Blue = 32809
ATIME = 255, Clear = 65535, Red = 30925, Green = 37940, Blue = 32619
ATIME = 255, Clear = 65535, Red = 30858, Green = 37867, Blue = 32572
ATIME = 255, Clear = 65535, Red = 30762, Green = 37813, Blue = 32547
ATIME = 255, Clear = 65535, Red = 30755, Green = 37780, Blue = 32499
ATIME = 255, Clear = 65535, Red = 30930, Green = 38015, Blue = 32690
ATIME = 255, Clear = 65535, Red = 30959, Green = 38012, Blue = 32670
ATIME = 255, Clear = 65535, Red = 31170, Green = 38315, Blue = 32915
Time 614.4ms: Mean=65535.0 | StdDev= 0.00 | Jitter=0.000%
=====
Part 2 Complete.

```

As we can observe from the above analysis, the output jitter is inversely proportional to the atime value, which is correct as initially the sensor is reading the values from 0 and the rise is substantial. Later, when the output nears the correct output range, it the curve starts flattening.

So, we can conclude the two by the following plot:



This plot correctly rectifies that the gain is not having much effect on the std dev and jitter, but the atime is having an inverse relationship with jitter and std dev.

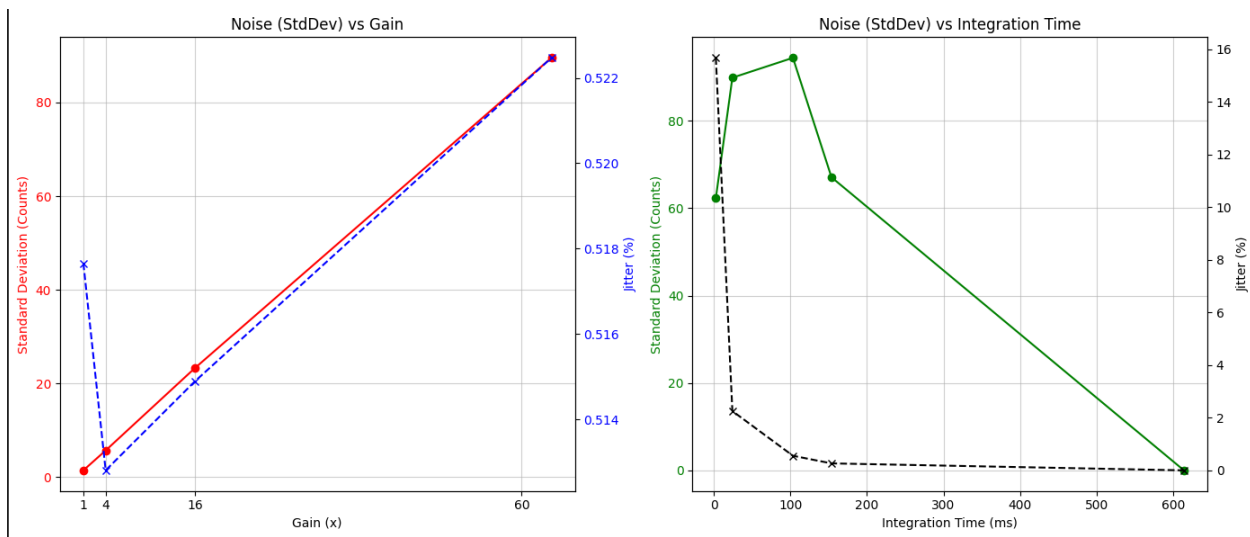
Summary

- Gain makes the numbers bigger but keeps the quality (SNR) the same.
- Integration Time actively improves the quality (SNR) by averaging out noise.
- Recommendation: For your final application, if you need precision, prioritize increasing ATIME (Time) over increasing AGAIN (Gain). Use Gain only if the signal is too dark to read.

Test 3.0:

Measure [mean, jitter] in dark sensor reading counts vs gain and vs atime.

Feature	Test 2.1 (Light)	Test 3.0 (Dark)
Source of Signal	Photons hitting the sensor (External).	Dark Current leaking inside the silicon (Internal).
Source of Noise	Shot Noise (Random arrival of photons). Noise grows as signal grows.	Thermal Noise (Electrons moving due to heat). Noise is constant (the "Noise Floor").
Expected Mean	High (e.g., 500 - 5000 counts).	Near Zero (0 - 5 counts) ¹ .
Expected Jitter	visible ($\sigma > 0$).	Likely 0.00 (suppressed by quantization) unless Gain is maxed.



Test 4.0:

4.0 verify colour sensor functionality (example: red colour light should output higher counts in red channel counts)

4.1 verify that relative intensity in the incident light (example $r:g:b \equiv 0.2:0.6:1$) is observed in the sensed readings . do the same vs atime and vs gain.

Test 5.0:

5.0 current , power consumed vs different sensor modes [running, idle, wait, sleep]

We did not perform this test because the power consumed reading that we were getting was very minimal that we were not able to make any reasoning for the same.

Test 6.0:

Determine the min and max I2C clock frequency for **reliable communication**.

To test it, we should call a connection between the Master and the Slave device, where the Master is the AD3 device and the slave is the TCS34725 sensor. This sensor is having a hard encoded number in the register address 0x12, which is the device ID.

- If we receive the "0x44" or "0x4d" for our sensor, the connection is proper.
- Otherwise, there is a problem in the bus.

Theoretically, the device supports the min and max freq of 0 and 400kHz respectively.

We can also test for the overclocking of the sensor, in order to test where does it's limit lies. Generally, it should be near 30% above the max limit for safety, but we have to test it.

```
print("---- Test 6.0: I2C Frequency Sweep (Min/Max Check) ----")

# 0Hz, 100Hz, 1kHz, 10kHz (Slow), 100kHz (Standard), 400kHz (Fast Mode Limit), 600kHz, 800kHz, 1MHz (Overclocking)
test_freq = [0, 100, 1000, 10e3, 50e3, 100e3, 200e3, 400e3, 500e3, 600e3, 800e3, 1000e3]

print(f'{"Frequency":<15} | {"Status":<10} | {"ID Read"}')
print('\n'+="*45)

pass_count = 0

for freq in test_freq:
    dwf.FDwfDigitalI2cRateSet(hdwf, c_double(freq))
    iNak = c_int()
    dwf.FDwfDigitalI2cClear(hdwf, byref(iNak))
    time.sleep(0.05)

    val = read_register(0x92)

    if val == 0x44 or val == 0x4D:
        status = "PASS"
        pass_count += 1
        id_str = f"0x{val:02X}"
    elif val is None:
        status = "FAIL"
        id_str = f"NACK (No Response)"
    else:
        status = "FAIL"
        id_str = f"CORRUPT (Read 0x{val:02X})"

    if freq>=1000:
        print(f'{freq/1000:8.0f} kHz | {status:<10} | {id_str}')
    else:
        print(f'{freq:9.0f} Hz | {status:<10} | {id_str}')

print("\n"+="*45)

dwf.FDwfDigitalI2cRateSet(hdwf, c_double(100e3))
print("Reset I2C Clock to standard 100kHz")
```

✓ 11.0s

```

--- Test 6.0: I2C Frequency Sweep (Min/Max Check) ---
Frequency      | Status      | ID Read
=====
    0 Hz | FAIL      | NACK (No Response)
   100 Hz | FAIL      | NACK (No Response)
    1 kHz | PASS      | 0x4d
   10 kHz | PASS      | 0x4d
   50 kHz | PASS      | 0x4d
  100 kHz | PASS      | 0x4d
  200 kHz | PASS      | 0x4d
  400 kHz | PASS      | 0x4d
  500 kHz | PASS      | 0x4d
  600 kHz | PASS      | 0x4d
  800 kHz | PASS      | 0x4d
 1000 kHz | PASS      | 0x4d
=====
Reset I2C Clock to standard 100kHz

```

As we can see here, the minimum frequency where we are getting a response is 1kHz, which is much larger than the provided 0Hz freq. Note that these are tested at the normal conditions (VDD=3V, TA=25C). But the interesting fact is that the returned code is 0x4D, which is for **TCS34727 sensor**, not the TCS34725 sensor.

The sensor cleared the maximum frequency test, but it failed the minimum frequency test as the minimum frequency here is 1kHz and highest here is 1000kHz.

Overclocking frequency:

```

print("--- Test 6.0: I2C Frequency Sweep (Min/Max Check) ---")

# 0Hz, 100Hz, 1kHz, 10kHz (Slow), 100kHz (Standard), 400kHz (Fast Mode Limit), 600kHz, 800kHz, 1MHz, 1.0e6, 1.2e6, 1.4e6, 1.6e6, 1.8e6, 2.0e6, 2.5e6 (Overclocking)
test_freq = [0, 100, 1000, 10e3, 50e3, 100e3, 200e3, 400e3, 500e3, 600e3, 800e3, 1000e3, 1.0e6, 1.2e6, 1.4e6, 1.6e6, 1.8e6, 2.0e6, 2.5e6]

print(f"{'Frequency':<15} | {'Status':<10} | {'ID Read'}")
print("\n"+"="*45)

pass_count = 0

for freq in test_freq:
    dwf.FDwfDigitalI2cRateSet(hdwf, c_double(freq))
    iNak = c_int()
    dwf.FDwfDigitalI2cClear(hdwf, byref(iNak))
    time.sleep(0.05)

    val = read_register(0x92)

    if val == 0x44 or val == 0x4D:
        status = "PASS"
        pass_count += 1
        id_str = f"0x{val:02x}"
    elif val is None:
        status = "FAIL"
        id_str = f"NACK (No Response)"
    else:
        status = "FAIL"
        id_str = f"CORRUPT (Read 0x{val:02x})"

    if freq>=1000:
        print(f"{freq/1000:8.0f} kHz | {status:<10} | {id_str}")
    else:
        print(f"{freq:9.0f} Hz | {status:<10} | {id_str}")

print("\n"+"="*45)

dwf.FDwfDigitalI2cRateSet(hdwf, c_double(100e3))
print("Reset I2C Clock to standard 100kHz")

```

```

--- Test 6.0: I2C Frequency Sweep (Min/Max Check) ---
Frequency      | Status      | ID Read

```

```

=====
      0 Hz | FAIL      | NACK (No Response)
     100 Hz | FAIL      | NACK (No Response)
      1 kHz | PASS      | 0x4d
     10 kHz | PASS      | 0x4d
     50 kHz | PASS      | 0x4d
    100 kHz | PASS      | 0x4d
    200 kHz | PASS      | 0x4d
    400 kHz | PASS      | 0x4d
    500 kHz | PASS      | 0x4d
    600 kHz | PASS      | 0x4d
    800 kHz | PASS      | 0x4d
   1000 kHz | PASS      | 0x4d
   1000 kHz | PASS      | 0x4d
   1200 kHz | PASS      | 0x4d
   1400 kHz | PASS      | 0x4d
   1600 kHz | PASS      | 0x4d
   1800 kHz | PASS      | 0x4d
   2000 kHz | PASS      | 0x4d
   2500 kHz | PASS      | 0x4d

```

```

=====
Reset I2C Clock to standard 100kHz

```

This tells us that the sensor is capable of much high levels of frequency than the company guarantees, as we can see here that even at 2500kHz we are able to observe the response.

Test 7.0:

7.0 & 7.1 usage of interrupt functionality

Verify that the sensor can "flag" you when the light level goes outside a specific range.

Extra Test Case-1:

* Provide a minimum of two novel test cases, including optional custom setup.